

Localized Artificial Intelligence and API Linux Setup



By Rui Yang

Purpose:

The purpose of this lab is to set up a locally hosted LLM that can both be accessed through the command-line interface and through a web page. It

should be able to run without access to the internet and accessible from anyone that is connected to the local network.

Background Information:

Machine learning: A branch of AI that focuses on teaching computers to learn from data without having to specifically program every rule; feeding a computer a large amount of data and let it find patterns and relationships on its own and use what it learned to make predictions and responses to new data or inputs.

Generative Artificial Intelligence: A subfield of artificial intelligence that uses generative models to produce text, images, videos, audio, software code, etc. These models learn and use their training data to produce an output based on a user input, oftentimes in the form of natural language prompts, which could be used as training data for the AI.

A large language model (LLM) is considered as a subsection of Generative AI, and it is a language model trained with self-supervised machine learning on a vast amount of text, designed for natural language processing tasks, especially language generation. The largest and most capable LLMs are generative pre-trained transformers (GPTs) and provide the core capabilities of chatbots.

Differences between a local LLM and a cloud-hosted LLM:

Locally hosted LLMs do not require internet access and can be accessed by anyone on the local network. This means that no data that goes into the AI goes onto the internet, and everything runs locally. This can be used to prevent data leaks from users entering private information into a cloud-hosted LLM and having the data routed across the internet and the company behind the cloud-hosted LLM having access to private and confidential information. However, local LLMs require a large amount of processing power, so to guarantee speed and accuracy, a large initial funding for semi-database level equipment is required to run the local LLM.

Docker is a platform used to run and manage applications in lightweight and standalone, isolated environments called containers, which contain everything needed to run the application, enabling better compute density and consistent deployment across different environments. The containers include the application code, libraries, settings, and can run regardless of the underlying infrastructure. Instead of virtualizing hardware to run different

operating systems, like a virtual machine, docker's containers are isolated just run on the hardware itself.

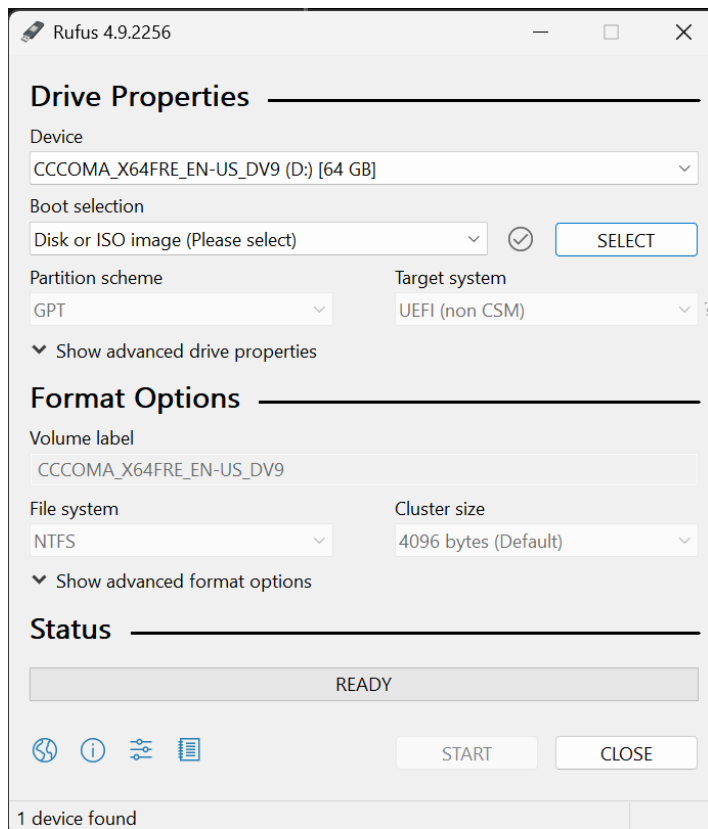
OpenWebUI is a web-based graphical interface that allows users to communicate with large language models (LLMs). Instead of using CLI prompts or API calls, users can access the AI through a browser, making it easier to test prompts, view responses, and manage conversations.

Ollama is the local large language model (LLM) host used in this lab. It enables open-source AI models such as Llama, GPT-OSS, or Gemma to directly run on a local computer rather than depending on remote cloud servers. Ollama manages downloading and loading these models so that OpenWebUI can display and interact with them in real time.

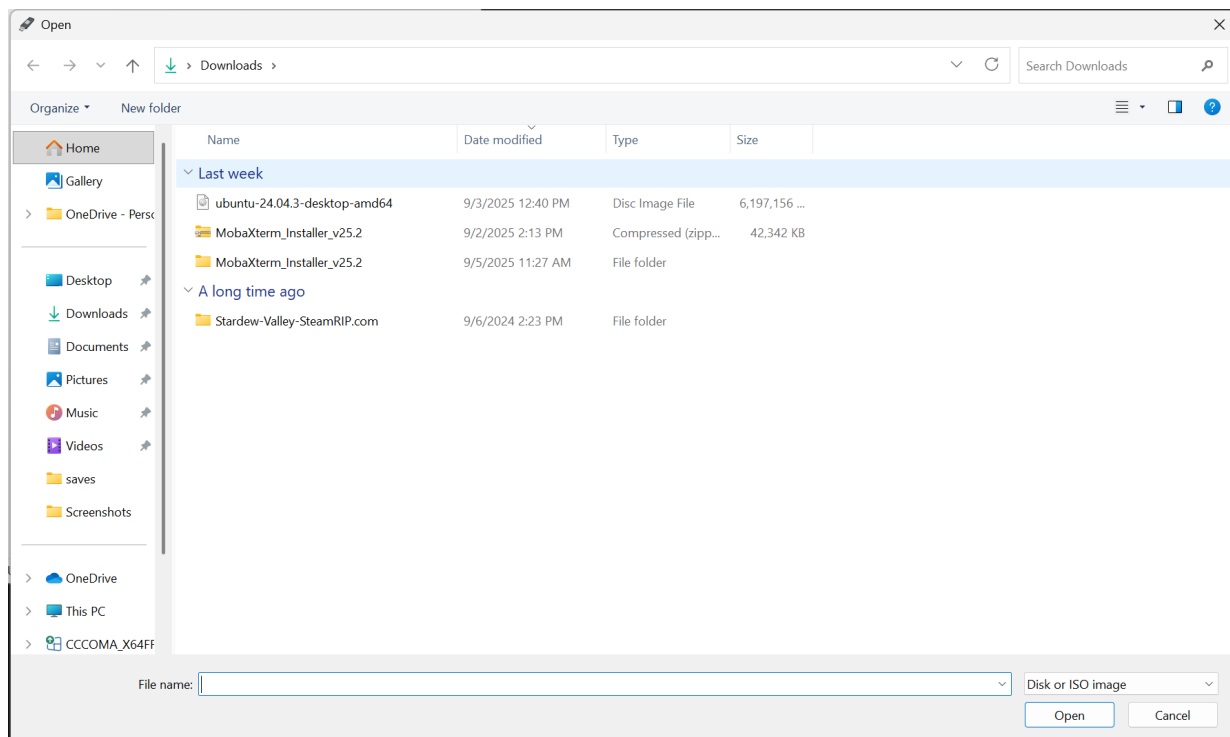
Lab Summary:

In this lab, we set up and ran a local large language model (LLM) using a combination of Docker, OpenWebUI, and Ollama. We establish a localized ai without the need to access the internet and being able to secure all information inputted in the AI. We also managed to input knowledge bases into the AI to have it respond according to trustworthy sources.

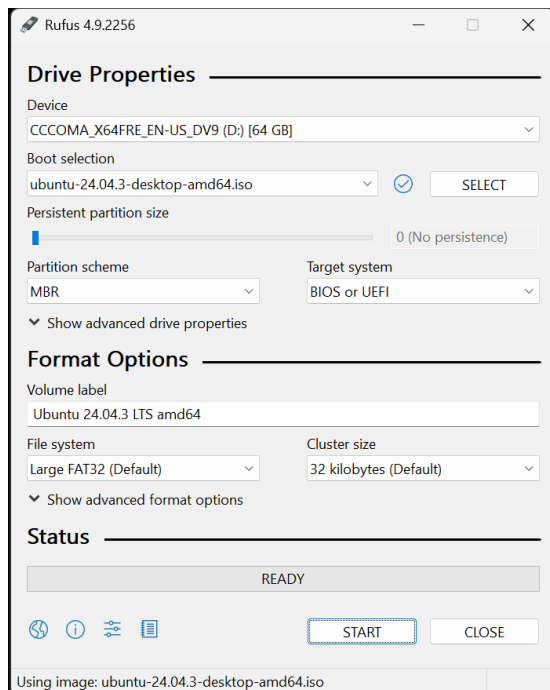
Use Rufus to install Ubuntu onto a USB to make a bootable USB:



Choose the ubuntu OS previously downloaded:



Press Start:



Leave all options as default and press ok, then choose to delete all partitions/data inside the drive.

ISOHybrid image detected



The image you have selected is an 'ISOHybrid' image. This means it can be written either in ISO Image (file copy) mode or DD Image (disk image) mode.

Rufus recommends using ISO Image mode, so that you always have full access to the drive after writing it.

However, if you encounter issues during boot, you can try writing this image again in DD Image mode.

Please select the mode that you want to use to write this image:

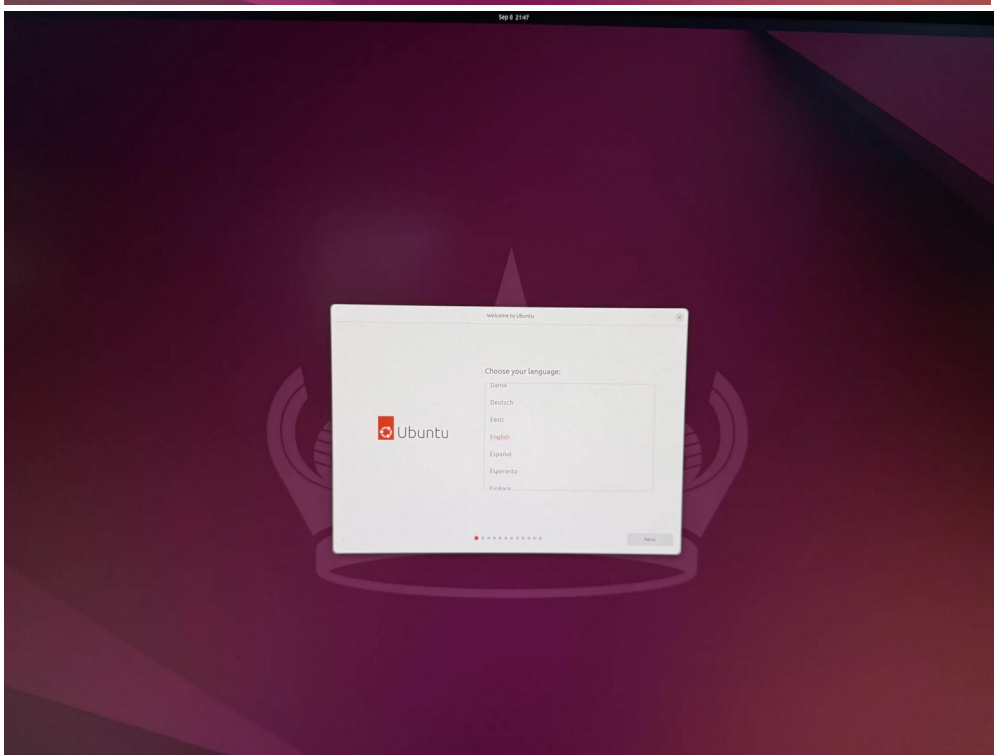
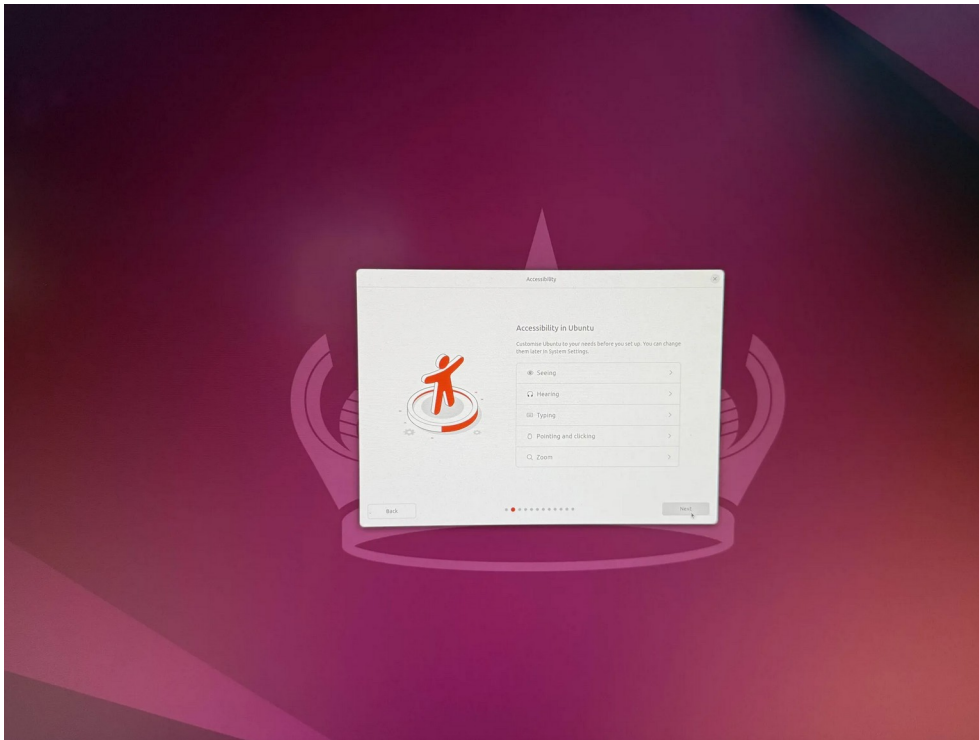
☒ Write in ISO Image mode (Recommended)

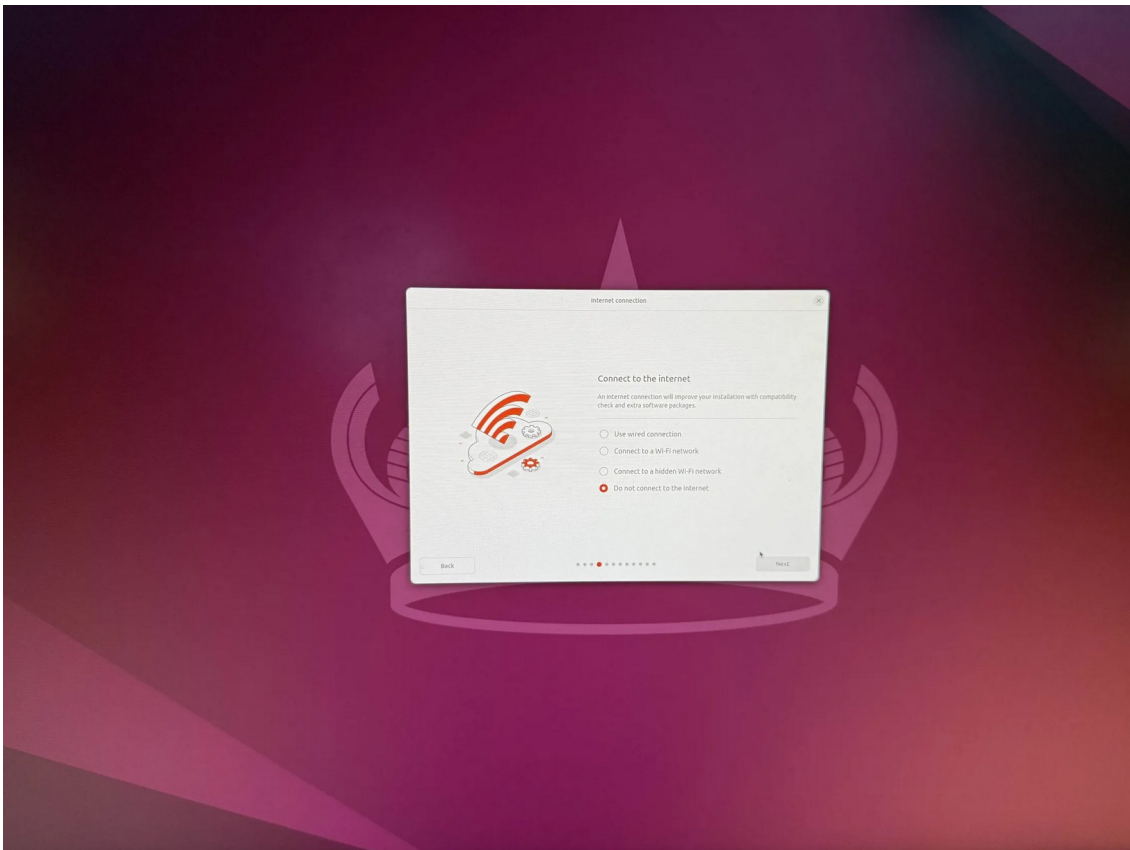
☐ Write in DD Image mode

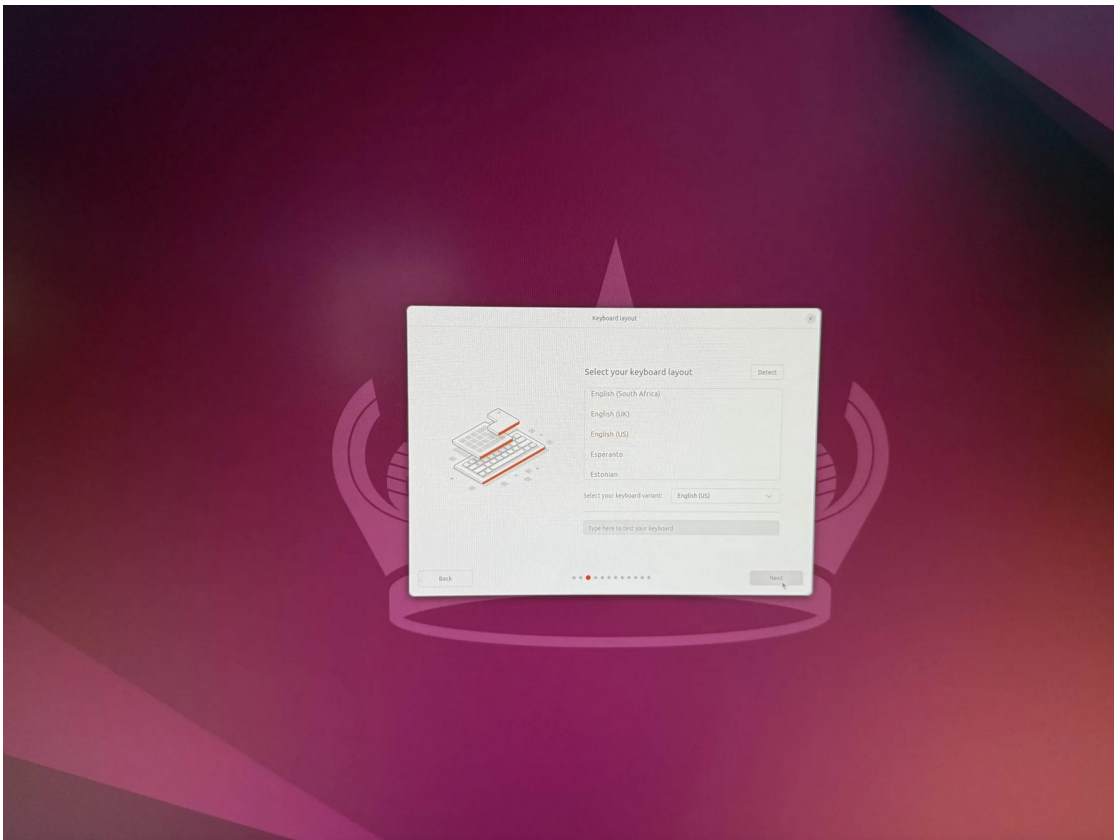
OK

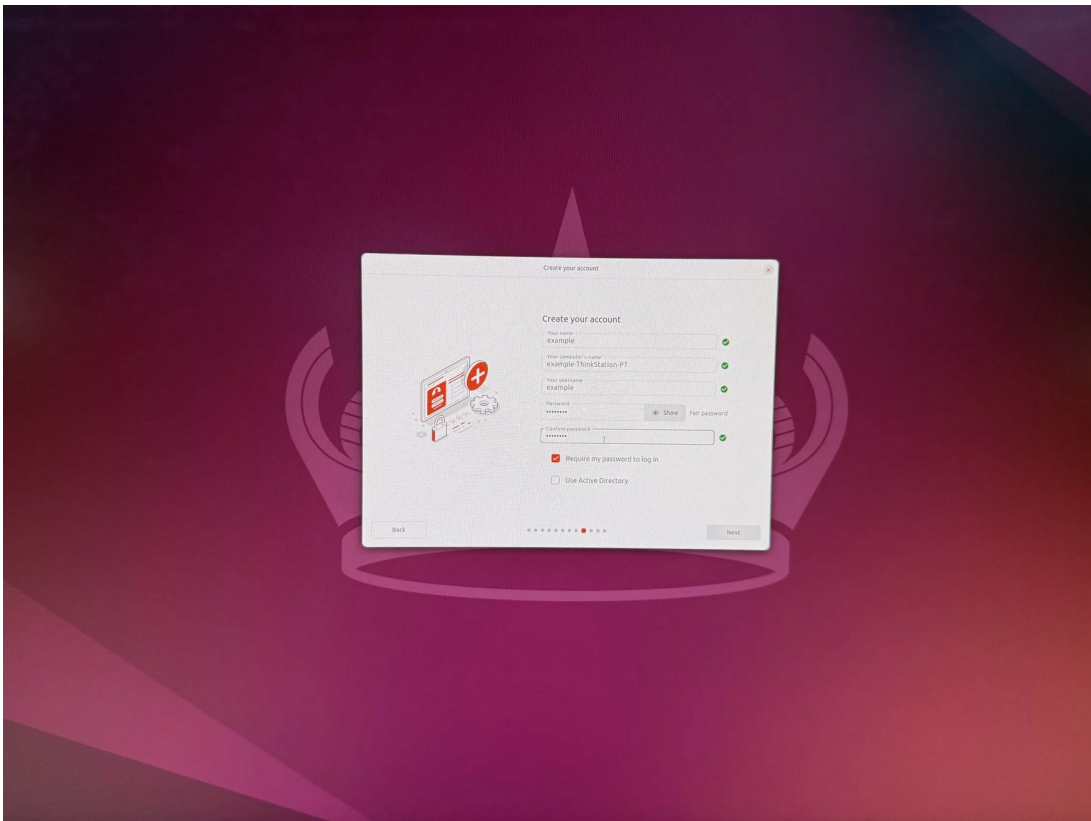
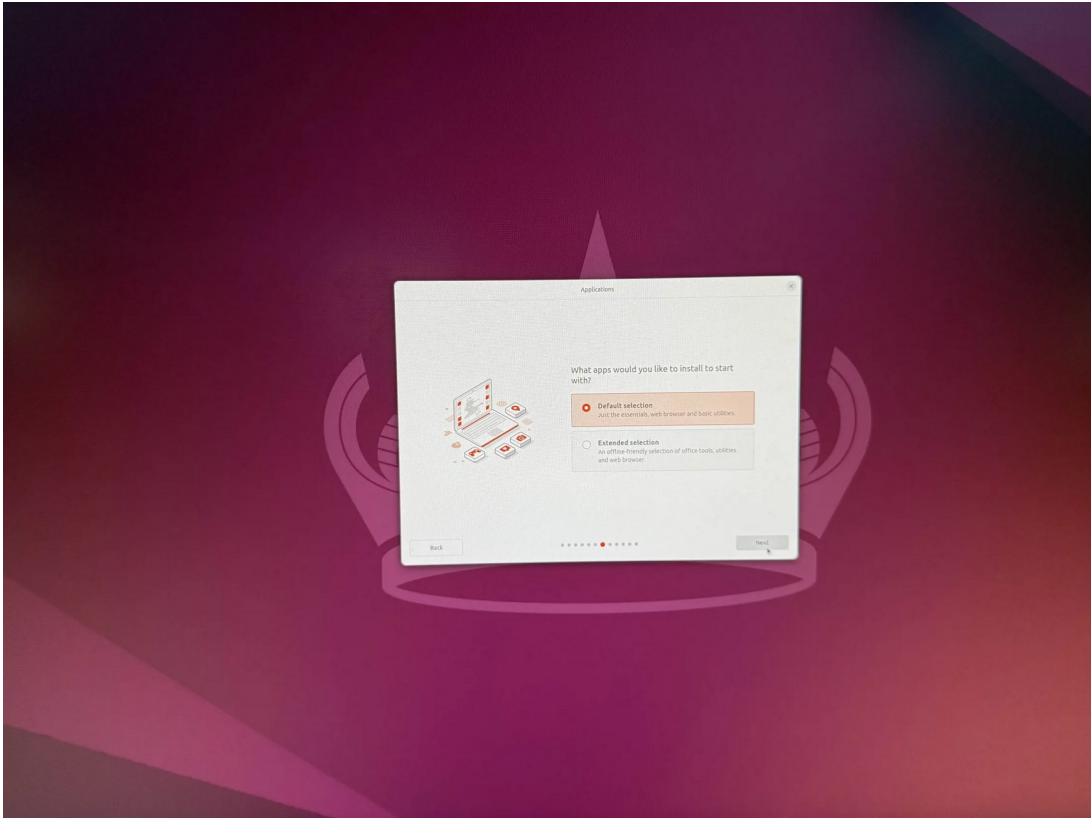
Cancel

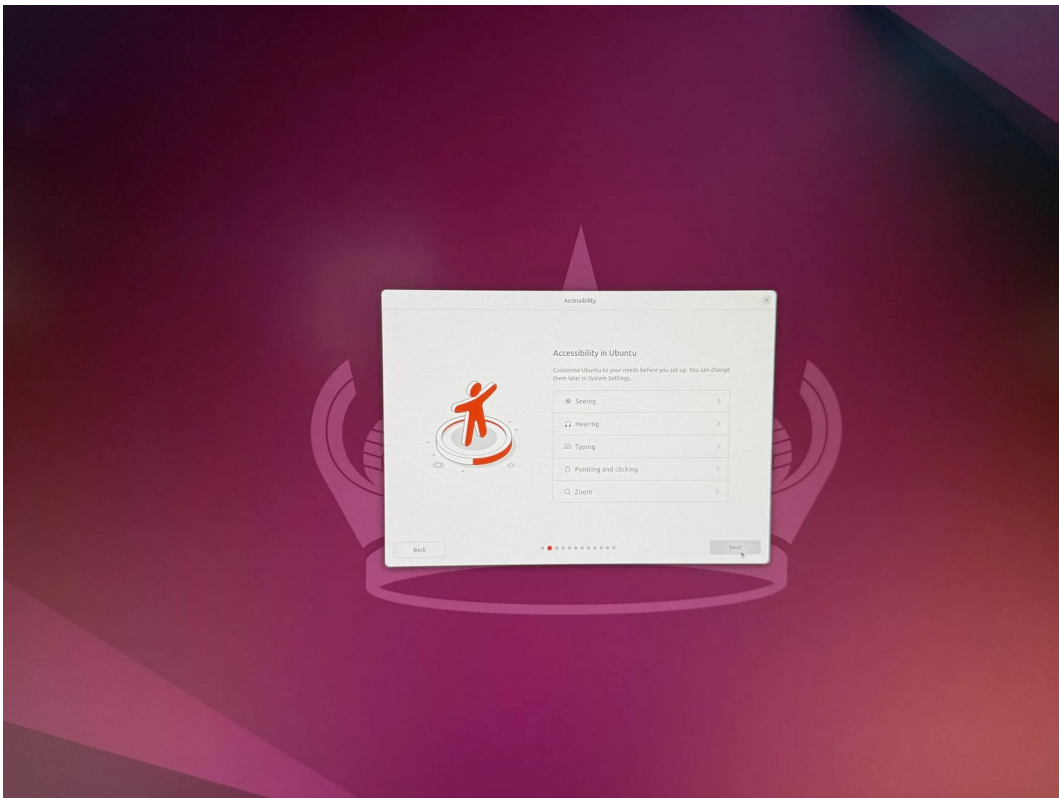
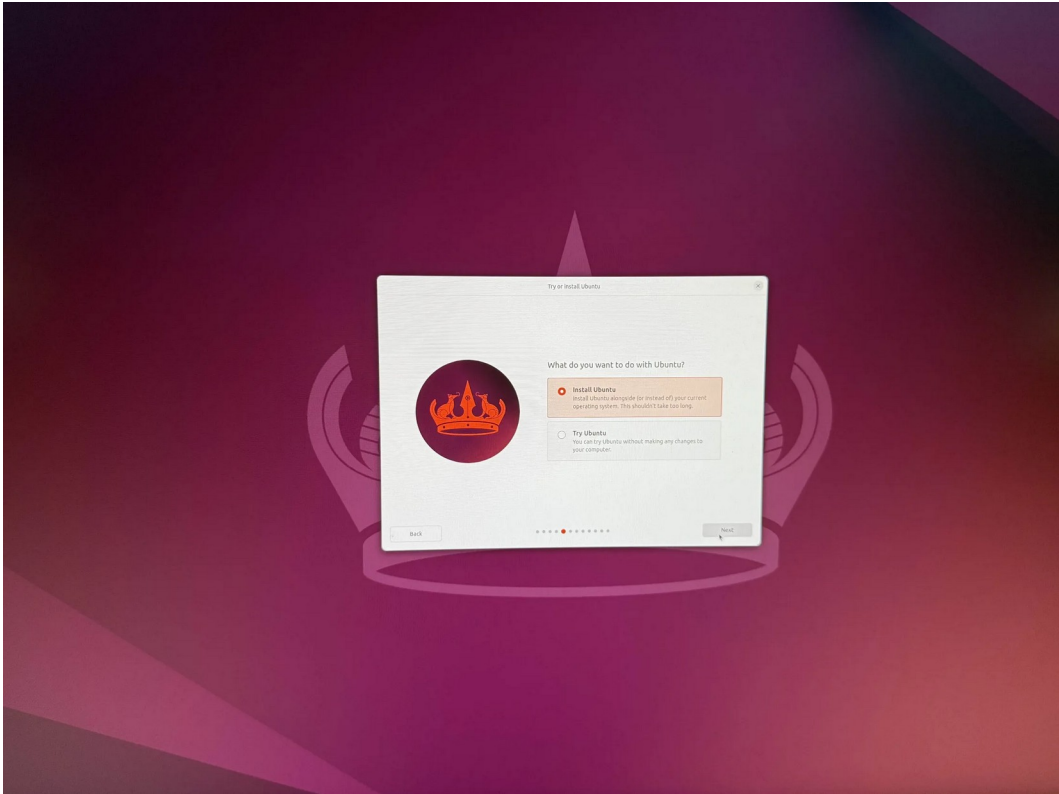
Boot from the bootable USB and go through the Ubuntu Installation sequence and finish installing ubuntu with the built in instructions.

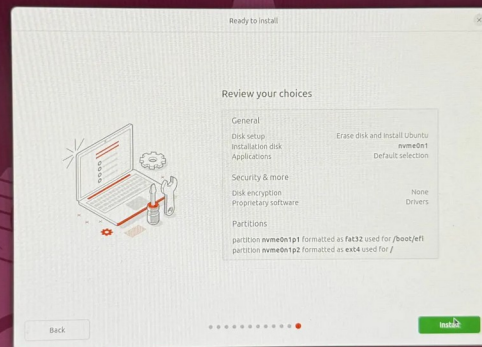


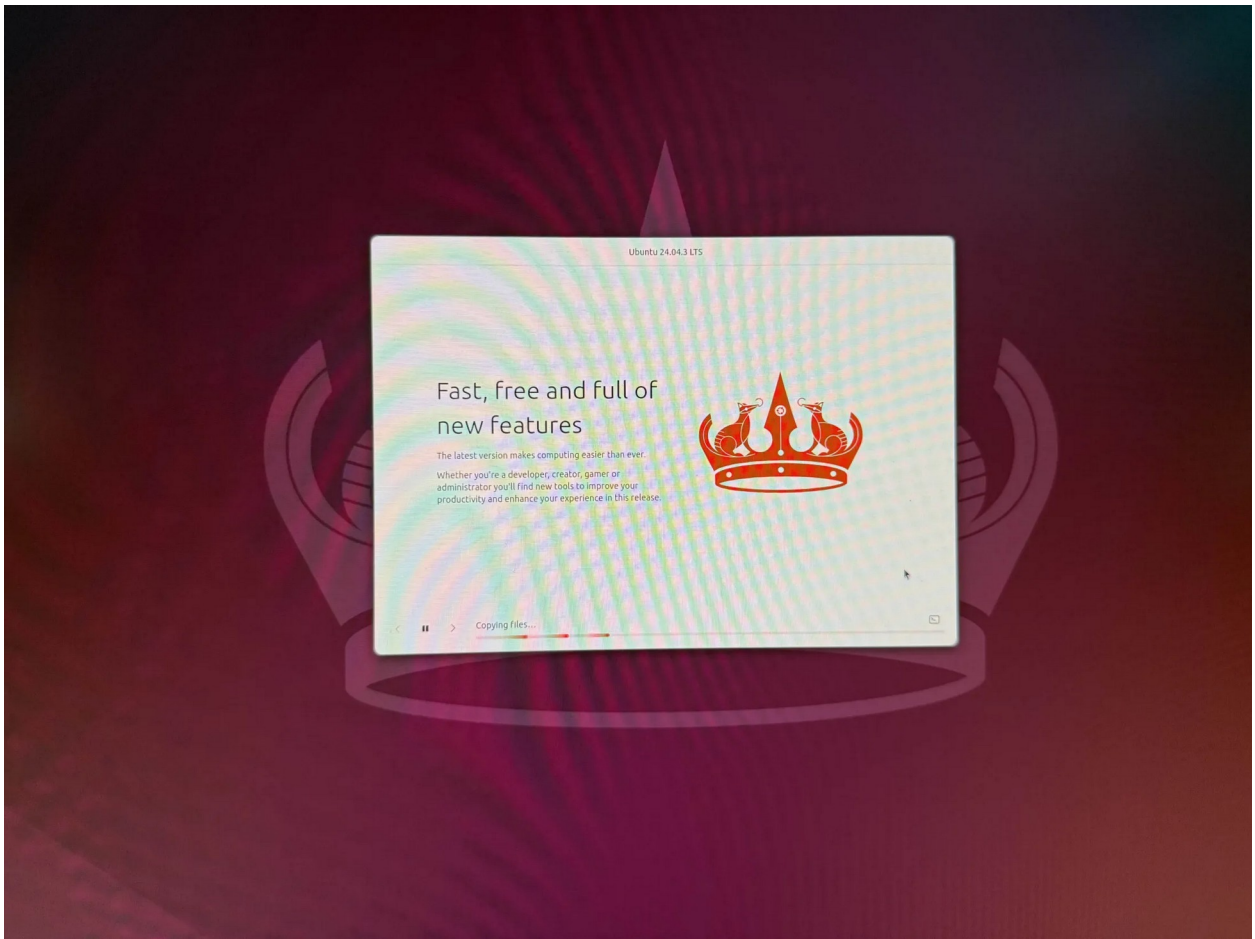




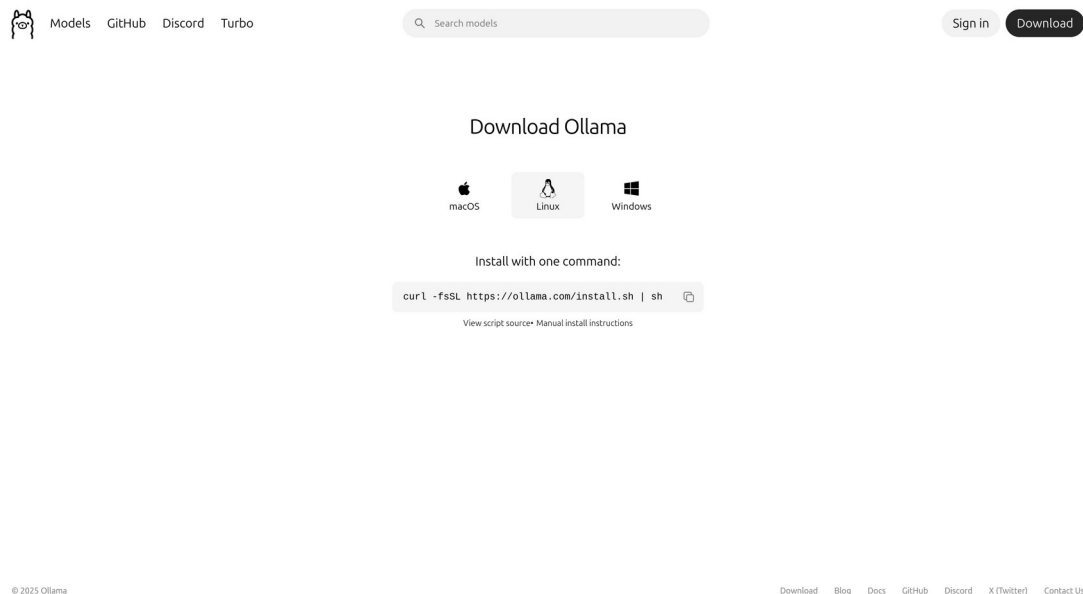






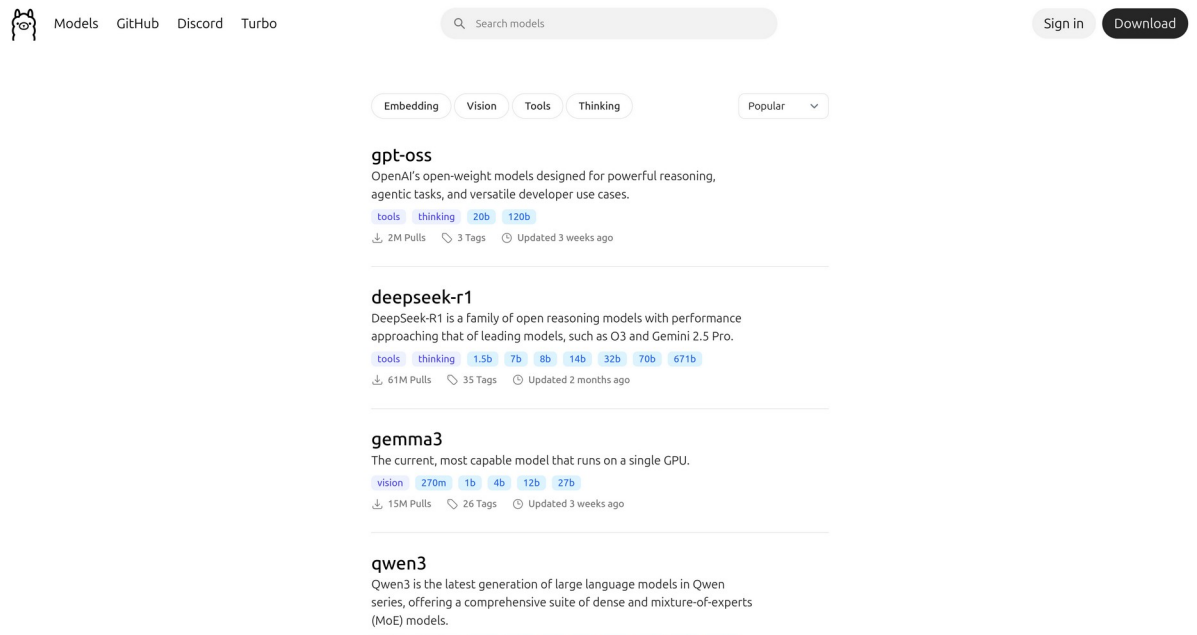


After successfully installing linux, go to the web browser and download ollama at [ollama download](https://ollama.com/download) with this command:



After installing ollama, look through the available ai models offered at ollama

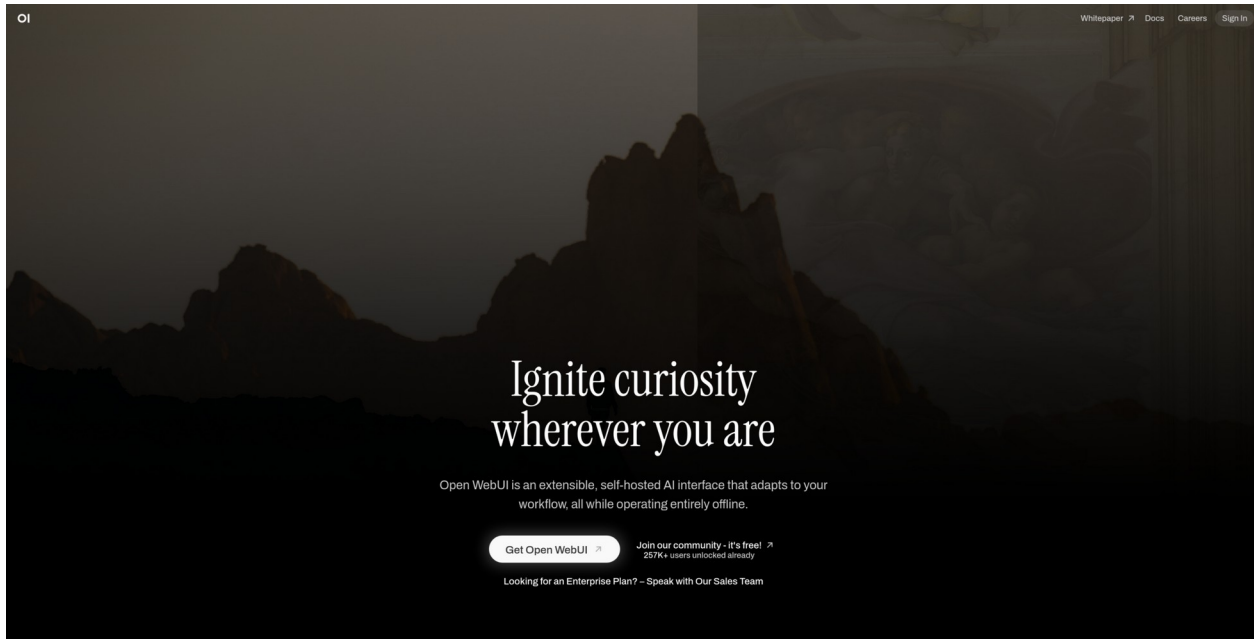
and choose one from the [search](#) feature on ollama that can be run off your v-ram.



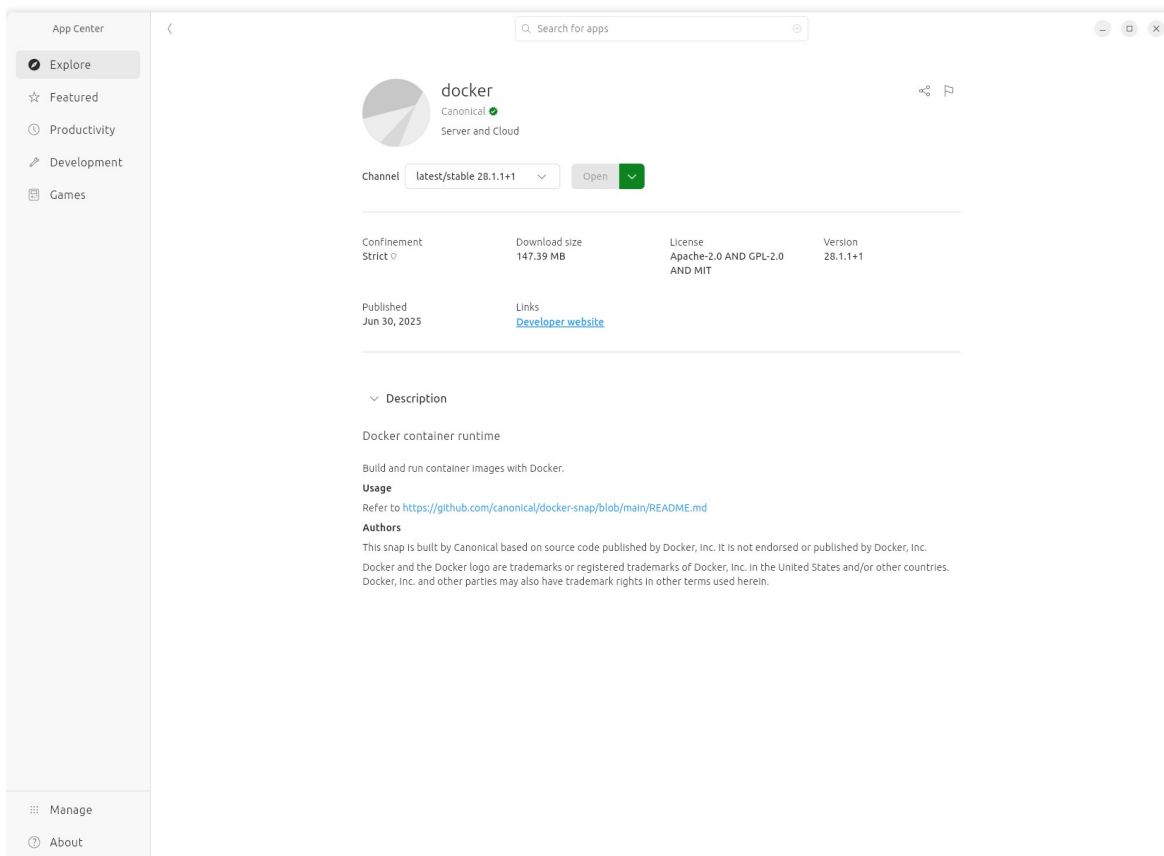
Download the AI model through terminal using the command “ollama run (ai model)”, an example is shown:

```
ollama run gemma3
pulling manifest
pulling aeda25e63ebd: 0% |
pulling manifest
pulling aeda25e63ebd: 1% | 46 MB/3.3 GB 2.9 MB/s 18m56s
```


Install OpenWebUI through their official website and open the [doc](#) located on top right:

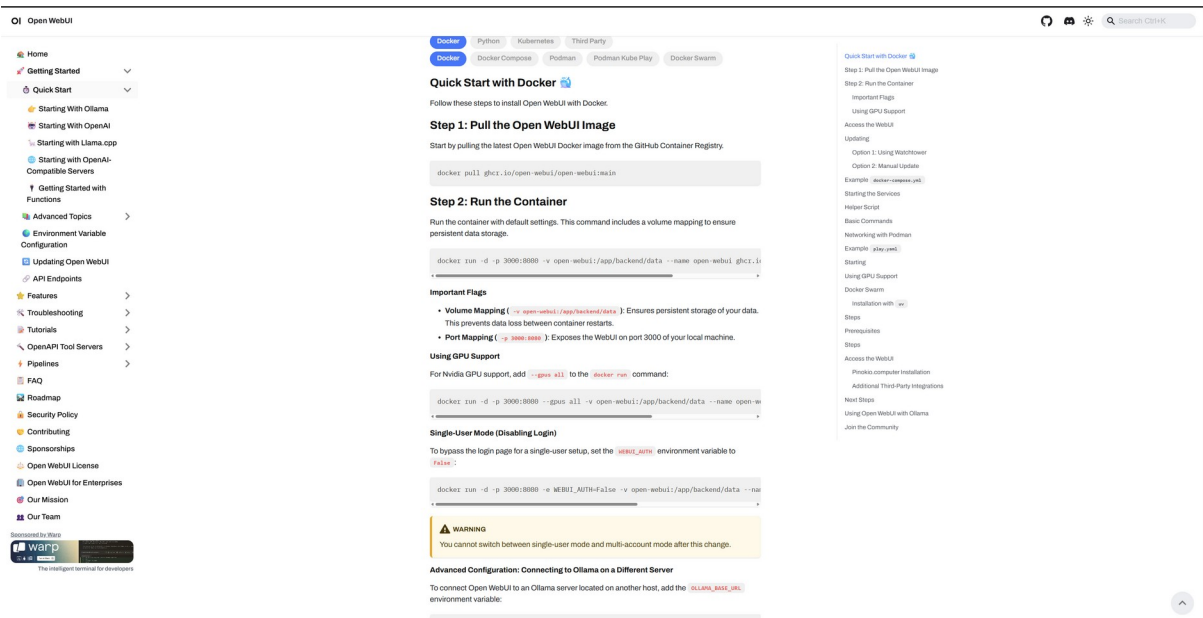


Install Docker on your machine through the App Center -

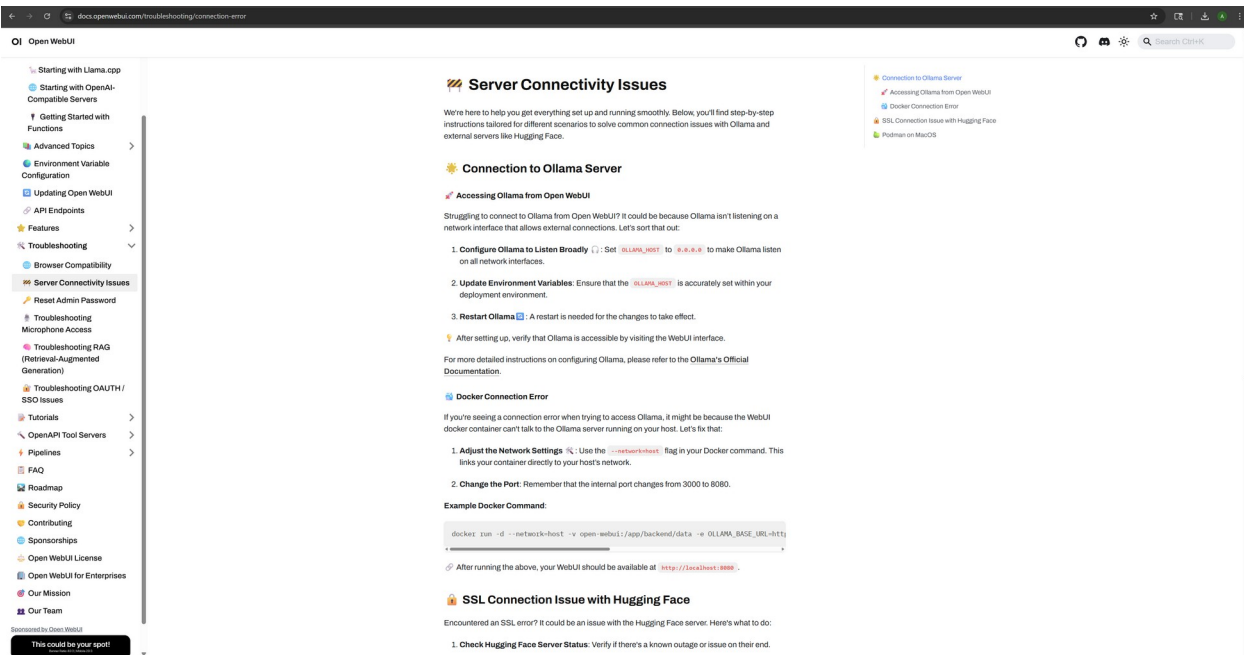


Navigate to the “Quick start” tab inside the docs and follow the instructions to

download OpenWebUI



Run the container with the “-network=host” flag in order to guarantee network connectivity. You can find the command inside the troubleshooting tab, under “Server Connectivity Issues”.

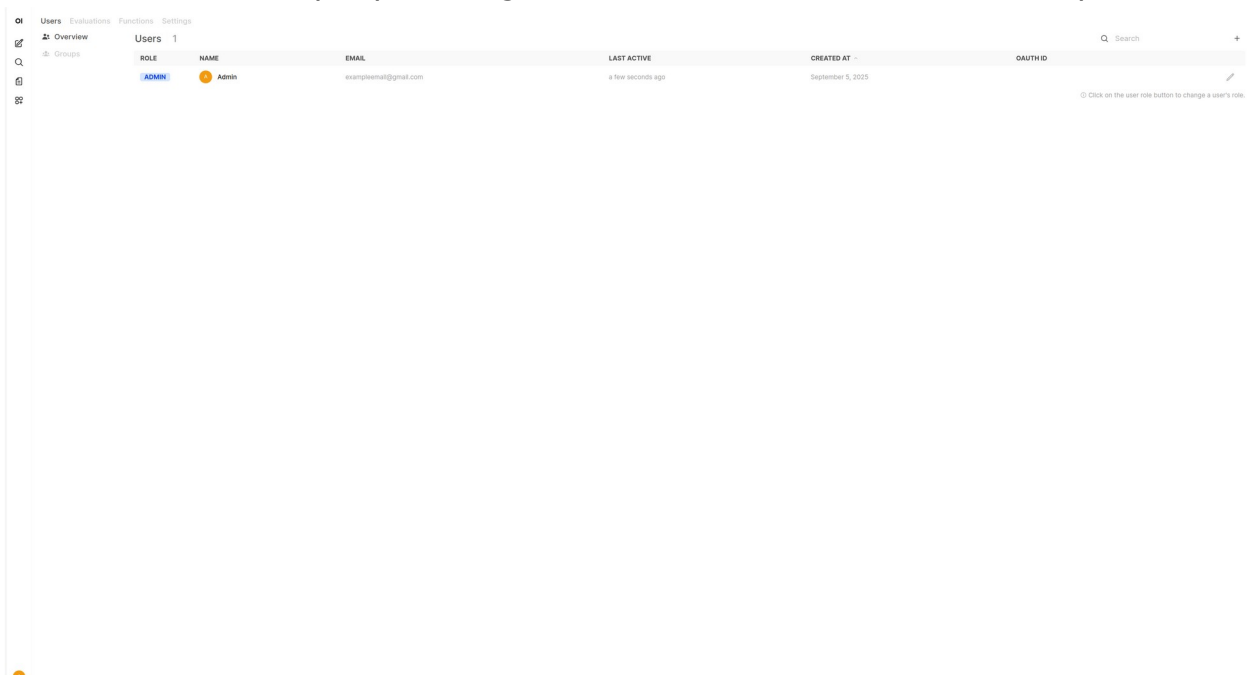


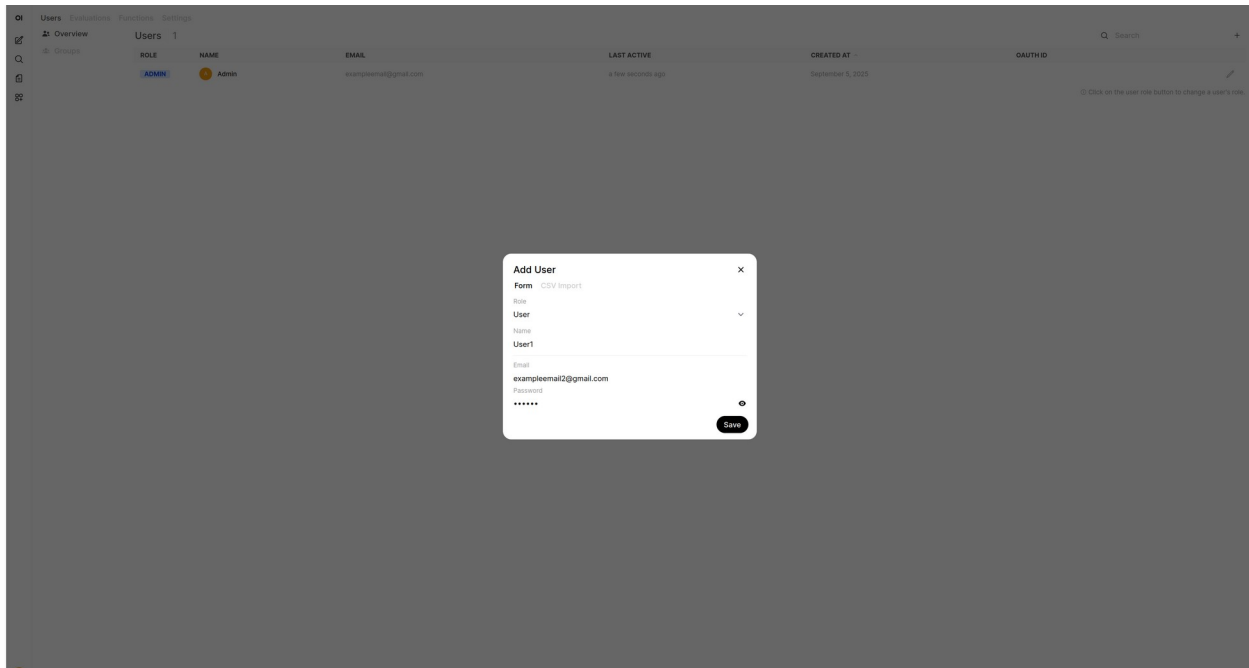
Access your WebUi at “private.i.p.address:8080” and set up your admin

account with your email and password, then navigate to the admin panel.

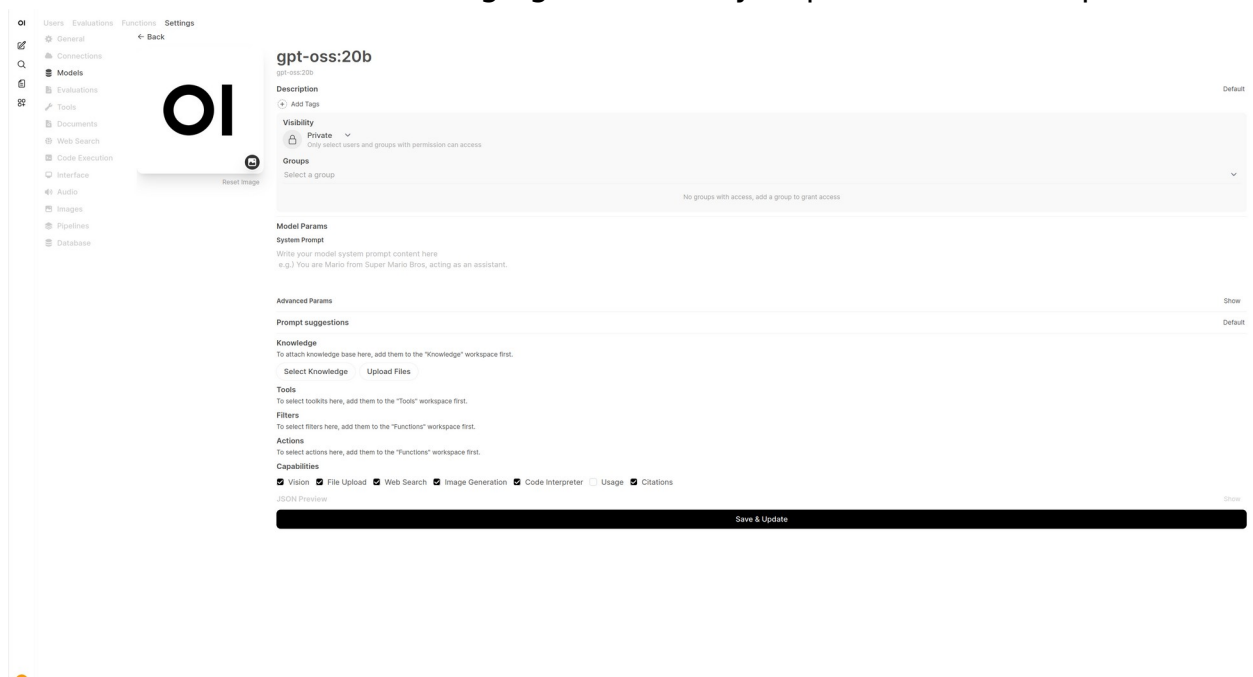


Create Users for the people using this service with the “+” on the top left





Navigate to the "Settings" bar at the top, then choose "models", and edit the current available models, changing the visibility to public instead of private



Everything is now complete!

Problems:

As this was a newly introduced concept, we started with a very basic knowledge of how LLMs functioned, but we needed to discover open-sourced

LLMs that were accessible to us and learned how to host them locally on our PCs. There were also some issues with running docker on Linux, where it wouldn't be able to run the OpenWebUI page, even though all the configurations were correct. There was also an underlying requirement for the OpenWebUI to be able to access ollama services and pull AIs, which was to have ollama downloaded and running on the PC at the same time.

Conclusions:

We were able to host a localized LLM that was accessible across the local network. It was able to run offline, and all the information it was using was saved locally without being sent to other companies. We also learned how to input additional information for the AI to use as context to answer some of the questions more accurately and truthfully. With a machine that has enough processing power and speed, we would be able to host a good local AI with a large training dataset and enough trustworthy context for it to use, we can reliably ask it questions and use it to help us troubleshoot and compile data.